

EV3 Line Follower – Assignment 2 Guide

30 September 2026

1 Background

This work controls a LEGO EV3 robot on yellow tape over a dark surface. A reflected-light color sensor supplies three states (WHITE, MIDDLE, BLACK); a Q table selects forward, left, or right actions. An infrared sensor detects obstacles. Q learning handles local steering; a junction route needs an explicit policy or extra sensing.

2 Files and duties

`safe_main.py` Final controller: motors A/D, color S1, IR S4, Q-table line following, Center stop, obstacle beep/reverse/turn/path reacquisition, and training mode.

`guided_calibration.py` Records 20 reflection samples for dark background, yellow tape, and tape edge; writes `calibration.json`.

`check_motor_direction.py` Pulses A and D with wheels lifted to verify wiring and direction.

`make_qtable_ev3.py` Converts the readable 18-entry seed to `q_table.pkl`.

`q_table_simulated.txt` Human-readable seed table.

`EV3_Line_Follower_Complete_Guide.pdf` Reference guide; boot files are retained but never flashed.

3 Code and training

The controller class reads reflection, maps it to WHITE/MIDDLE/BLACK, executes motor commands and stops on Center or an obstacle, and brakes both motors during cleanup. `best_action` selects the highest Q value for $(mode, state)$. On an obstacle the robot beeps, reverses, pivots without RL, accepts stable WHITE or MIDDLE as a recovered path, beeps again, and resumes. Training observes a transition, gives +10 for MIDDLE and -10 otherwise, applies $Q \leftarrow Q + \alpha(r + \gamma Q'_{max} - Q)$, and saves `q_table_candidate.pkl`. Keep a copy of the seed table before training.

4 Complete commands

Use the current Bluetooth address shown by the EV3. Open the terminal in the folder containing these files. The commands below use WSL syntax.

```

EV3_IP=169.254.73.240
# Computer (WSL)

ping $EV3_IP
ssh robot@$EV3_IP                                # password: maker

# EV3 SSH shell
mkdir -p /home/robot/assignment2
exit

# Computer: upload
scp safe_main.py guided_calibration.py check_motor_direction.py make_qtable_ev3.py
ssh robot@$EV3_IP
cd /home/robot/assignment2

# EV3: build, calibrate, motor test
brickrun -r — pybricks-micropython make_qtable_ev3.py
brickrun -r — pybricks-micropython guided_calibration.py
brickrun -r — pybricks-micropython check_motor_direction.py

# Normal run (edit safe_main.py so TRAINING = False)
brickrun -r — pybricks-micropython safe_main.py
# Center starts and stops

# Training (backup, edit TRAINING = True, then run)
cp q_table.pkl q_table_seed_backup.pkl
nano safe_main.py
brickrun -r — pybricks-micropython safe_main.py
# candidate: q_table_candidate.pkl; Center starts/stops

# Retrieve after exit
exit
scp robot@$EV3_IP:/home/robot/assignment2/calibration.json .
scp robot@$EV3_IP:/home/robot/assignment2/q_table_candidate.pkl .
scp robot@$EV3_IP:/home/robot/assignment2/*.log .

```

5 Motor EIO error

For OSError: [Errno 5] EIO on line 9, inspect ports:

```
for d in /sys/class/tacho-motor/*; do echo "$d"; cat "$d/address"; cat "$d/driver_n
```

Both outA and outD must report lego-ev3-1-motor. Power off, reconnect cables, reboot, and test independently:

```
brickrun -r — pybricks-micropython -c "from pybricks.ev3devices import Motor; from
brickrun -r — pybricks-micropython -c "from pybricks.ev3devices import Motor; from
```